



Project 4: Machine Learning



1. Project Overview
2. Core Objectives
3. Technical Requirements & Environment
4. Project Tasks
5. Graduate Extension
6. Deliverables
7. Grading Policy
8. Use of AI Tools

Project 4: Cloud-Native ML Deployment

Release Date: Thursday, April 30, 2026

Due Date: Thursday, May 14, 2026, 11:59 PM EST

1. Project Overview

This project provides hands-on experience in the operational aspects of machine learning by challenging you to deploy a pre-trained model as a scalable, production-ready web service. Instead of focusing on model creation, you will architect a robust inference system using cloud-native components. You will select a pre-trained image classification model, deploy it using a managed ML service, and expose it through a serverless API. The goal is to understand the architecture, trade-offs, and best practices for serving ML models in the cloud, emphasizing reliability, scalability, and cost-efficiency.

2. Core Objectives

- Deploy a pre-trained machine learning model on a managed cloud platform.
- Build a secure and scalable serverless API to serve real-time predictions.
- Integrate cloud storage for managing model artifacts.
- Understand the end-to-end architecture of a modern ML inference system.
- Analyze the performance (latency) and estimate the operational cost of your cloud-based solution.

Chat Assistant



- Gain practical experience with key services from a major cloud provider (AWS, Google Cloud, or Azure).

3. Technical Requirements & Environment

- **Cloud Platform:** You must use one of the following major cloud providers:
 - **AWS:** AWS SageMaker, Amazon S3, AWS Lambda, Amazon API Gateway.
 - **Google Cloud:** Google Vertex AI, Google Cloud Storage, Google Cloud Functions, Google API Gateway.
 - **Azure:** Azure Machine Learning, Azure Blob Storage, Azure Functions, Azure API Management.
- **ML Model:** Use a **pre-trained** image classification model. You are **not** required to train a model. Excellent sources include:
 - **TensorFlow Hub:** Provides a wide variety of models in SavedModel format.
 - **PyTorch Hub:** Offers numerous pre-trained models.
 - Framework-native libraries like `torchvision.models` or `tensorflow.keras.applications`.
 - **Recommended Models:** MobileNetV2 or ResNet50 are excellent choices as they are well-documented and widely supported.
- **Dataset for Testing:** No training dataset is required. You will need a few sample images (e.g., a cat, a dog, a car) to test your final API endpoint. You can source these from the internet.
- **Development Environment:** You will write a serverless function in a supported language (Python is highly recommended). Your code will interact with cloud provider SDKs (e.g., `boto3` for AWS, `google-cloud-aiplatform` for GCP, `azure-ai-ml` for Azure).

4. Project Tasks

Task 1: Model Preparation and Cloud Storage

1. **Select and Download a Model:** Choose a pre-trained image classification model from a source like TensorFlow Hub or PyTorch Hub. Download the model artifacts. For TensorFlow, this will typically be a `SavedModel` directory. For PyTorch, a `.pth` file.
2. **Package Model Artifacts:** Ensure your model artifacts are in the correct format for your chosen cloud's ML service. Sometimes this requires creating a `.tar.gz` archive containing the model and an inference script. Research the specific requirements for SageMaker, Vertex AI, or Azure ML.
3. **Upload to Cloud Storage:** Create a storage bucket (S3, GCS, or Azure Blob) and upload your packaged model artifacts to it. This bucket will be used for inference.

Chat Assistant

service deploys the model.

Task 2: Deploying the Model as a Managed Endpoint

1. **Configure Deployment:** Using your chosen cloud's MLaaS platform (SageMaker, Vertex AI, or Azure ML), define a new model resource, pointing it to the model artifacts in your storage bucket.
2. **Create an Endpoint:** Configure and deploy the model to a real-time inference endpoint. This step involves selecting the appropriate instance type for serving (a small CPU-based instance is sufficient and cost-effective) and specifying the container image for the model server (many are pre-built by the cloud provider).
3. **Test the Endpoint:** Once the endpoint is active, use the cloud provider's SDK or CLI to send it a sample request to ensure it is working correctly before moving to the next task. This is a critical debugging step.

Task 3: Building a Serverless API Endpoint

1. **Implement a Serverless Function:** Write a cloud function (Lambda, Google Cloud Function, or Azure Function) that will serve as the public-facing interface for your service. This function's code should:
 - Accept an HTTP request containing an image (e.g., a URL to an image or a base64-encoded image string in a JSON payload).
 - Preprocess the input image into the format expected by your model (e.g., resize to 224x224, normalize pixel values).
 - Invoke the managed ML endpoint you created in Task 2, passing the preprocessed data.
 - Receive the prediction from the endpoint.
 - Post-process the result into a human-readable JSON format (e.g., `{"prediction": "golden_retriever", "confidence": 0.92}`).
 - Return this JSON response to the client.
2. **Expose via API Gateway:** Configure an API Gateway to trigger your serverless function via a public HTTP endpoint (e.g., `POST /classify`). This provides a stable, secure, and scalable entry point for users.

Task 4: System Analysis and Documentation

1. **Performance Testing:** Test your final API Gateway endpoint. Use a tool like `curl` or Postman to send several requests with different images.
 - Measure the end-to-end latency for at least 10 different requests and report the average.End-to-end latency is the time from when you send the request to when you receive the final response.

Chat Assistant

- In your report, discuss factors that might contribute to this latency (e.g., network, function cold starts, model inference time).

2. Scalability & Cost Analysis:

- **Scalability:** You do not need to perform a large-scale load test. Instead, in your report, *discuss* the scalability of your architecture. Why are serverless functions and managed ML endpoints considered scalable? What component would be the first to break under extreme load?
- **Cost:** Investigate the pricing models for the cloud services you used (API Gateway, Lambda/Cloud Functions, SageMaker/Vertex AI Endpoint, S3/Cloud Storage). Create a simple cost estimate for a hypothetical scenario of processing 100,000 images per month.

5. Graduate Extension

Undergraduate students may attempt this section for extra credit worth up to 10% bonus on the project. Graduate students are REQUIRED to complete this section and it is worth 10% of the total project grade.

Task 5: Scalability or Reliability Extension

Implement **one** meaningful extension beyond the core project that improves the production readiness of your ML inference system. Your extension must involve a real implementation and evaluation, not just a design discussion. You may choose from one of the following options:

1. **Autoscaling Experiment:** Configure autoscaling for your managed ML endpoint, serverless layer, or both. Demonstrate how the system responds to changing request volume, and analyze the effect on latency, throughput, and cost.
2. **Asynchronous Inference Pipeline:** Build an asynchronous version of the workflow using a queue, storage-triggered pipeline, or batch inference service. Explain when asynchronous inference is preferable to real-time serving.
3. **Monitoring and Reliability Enhancement:** Add production-oriented monitoring such as request/error metrics, logs, alerts, or failure handling for malformed inputs and upstream endpoint failures. Demonstrate at least one failure scenario and how your system detects or handles it.

For this extension, include implementation evidence, measurements or observations, and a short discussion of trade-offs.

6. Deliverables

Chat Assistant

Submit a **single PDF file** to the course portal. This document must include:

1. Report (PDF format): A comprehensive report detailing:

- **Introduction:** Briefly describe the project goals and your chosen cloud platform and services.
- **System Architecture:** Include a simple architecture diagram showing how the different cloud services (Storage, MLaaS, Serverless Function, API Gateway) connect and interact.
- **Implementation Details:** Describe the steps you took for each task. Highlight any challenges you faced and how you solved them (e.g., model packaging issues, IAM permissions, API configuration).
- **API Endpoint Instructions:** Provide clear instructions on how to use your API, including the URL and an example `curl` command with a sample request body.
- **Performance & Cost Analysis:** Present your latency measurements and provide the thoughtful analysis of scalability and cost as described in Task 4.
- **Graduate Extension (Graduate Students Required / Undergraduate Bonus if Attempted):** Describe the advanced extension you implemented, include evidence, and discuss the benefits and trade-offs.
- **Conclusion & Learnings:** Summarize your findings and key takeaways from building and deploying a cloud-native ML system.

2. Code:

- Append the complete, well-commented code for your serverless function to your report.
- Include a `requirements.txt` file listing any external libraries your function uses.

7. Grading Policy

Your project grade will be based on the following components:

- **[20 points] Task 1 & 2: Model Deployment**

- (5) Correct selection and packaging of a pre-trained model.
- (5) Successful upload and storage of model artifacts in the cloud.
- (10) Functional deployment of the model to a managed, real-time endpoint.

- **[30 points] Task 3: Serverless API Implementation**

- (15) Correct implementation of the serverless function, including data pre/post-processing and endpoint invocation logic.
- (10) Successful configuration of the API Gateway to expose the function as a public HTTP endpoint.
- (5) The final API is fully functional and returns predictions correctly.

Chat Assistant

- **[25 points] Task 4: System Analysis**

- (10) Thorough latency testing with results clearly presented in the report.
- (10) Insightful discussion of the system's scalability and a well-reasoned cost estimation.
- (5) Clear explanation of potential performance bottlenecks or trade-offs.

- **[25 points] Report and Code Quality**

- (10) **Report:** The report is well-organized, clearly written, and includes all required sections (especially the architecture diagram and analysis).
- (10) **Code:** The serverless function code is well-commented, structured logically, and easy to understand.
- (5) **Reproducibility:** The instructions for using the API are clear and accurate. A TA can easily test the provided endpoint.

- **[10 points] Graduate Extension / Undergraduate Bonus**

- (10) Successful implementation and evaluation of one advanced production-readiness extension beyond the core project.

- **Deductions:** Points may be deducted for late submissions according to the course policy, or for academic integrity violations.

Undergraduate Bonus: Undergraduate students who complete Task 5 can receive up to a **10% bonus** added to their final project score, for a maximum possible score of 110. Graduate students are graded out of the full 110 points and then normalized to 100.

8. Use of AI Tools

You are **ALLOWED** to use LLM-based AI assistants (e.g., ChatGPT, GitHub Copilot) as a supplementary tool. These can be helpful for generating boilerplate code, debugging specific errors, explaining configuration options, or exploring alternative approaches. However, you are responsible for validating the correctness, security, and applicability of any information or code provided by an LLM. Directly submitting unverified or misunderstood LLM output is not acceptable. You must be able to explain all parts of your final submission.

Chat Assistant